Software Engineering Workstation & Local Developer Environment Setup Guide

Version 4.2 • Maintainer: Developer Productivity & Core Data Platform Engineering • Target OS: macOS 15+, RHEL 9.4+, Windows 11 WSL2

Section 1. Workstation Baseline Requirements and MDM Enrollment

Every software engineer, data scientist, and site reliability engineer must configure their corporate-issued workstation in compliance with the Endpoint Engineering Standard before cloning internal source repositories or connecting to development databases. Personal (BYOD) laptops are strictly prohibited from storing source code, database credentials, or internal TLS certificates.

1.1 Mandatory Endpoint Protection and Disk Encryption

Before installing developer toolchains, verify that your workstation passes all four automated checks in the IBM Endpoint Self-Service Agent (`Mac@IBM` Self-Service on macOS, `OpenClient` Management on Red Hat Enterprise Linux, or `Win@IBM` Company Portal on Windows 11):

1. **Full-Disk Encryption:** Apple FileVault (macOS), LUKS2 AES-XTS-Plain64 512-bit encryption (RHEL 9), or BitLocker with TPM 2.0 PIN (Windows 11) must be active and escrow recovery keys synced to the Endpoint Vault.
2. **CrowdStrike Falcon Sensor:** Verify that the CrowdStrike Falcon daemon is running (`sudo /Applications/Falcon.app/Contents/Resources/falconctl stats` on macOS or `systemctl status falcon-sensor` on Linux).
3. **Temporary Elevated Privileges (`sudo` Access):** Standard engineering accounts run with standard user privileges by default. To execute privileged installation commands, launch the `IBM Privilege Elevation` tray utility, enter justification code `DEV-SETUP-LOCAL`, and obtain a 30-minute local `sudo` session.
4. **Internal Root Certificate Authority Bundle:** Install the corporate internal PKI Root and Intermediate CA bundle so CLI tools (`curl`, `pip`, `npm`, `git`, and `docker`) trust internal HTTPS endpoints (`*.ibm.internal`). Execute the automated trust bootstrap script:

```
# Bootstrap IBM Internal PKI Root Certificates into OS and Python/Node trust
stores
curl -fsSL https://pki.ibm.internal/bootstrap/install-ibm-ca.sh | sudo bash

# Verify environment variables exported in ~/.zshrc or ~/.bashrc:
export REQUESTS_CA_BUNDLE="/etc/ssl/certs/ibm-internal-ca-bundle.pem"
export NODE_EXTRA_CA_CERTS="/etc/ssl/certs/ibm-internal-ca-bundle.pem"
export SSL_CERT_FILE="/etc/ssl/certs/ibm-internal-ca-bundle.pem"
```

Section 2. Core SDKs, Language Runtimes, and Internal Artifact Mirrors

Direct downloads from unverified public package registries are rate-limited and audited by corporate proxy filters. All developers must configure their local package managers to pull dependencies through the internal **Artifactory Enterprise Mirror (`artifactory.ibm.internal`)**, which performs automated CVE vulnerability scanning and license compliance validation.

2.1 Python 3.11 / 3.12 Environment and PyPI Mirror Configuration

All AI, RAG, and backend data services standardize on **Python 3.11.9** or **Python 3.12.4** managed via `pyenv` and `poetry` (or `uv`). Configure `~/.pip/pip.conf` to authenticate against the internal Artifactory PyPI proxy:

```
# Create ~/.pip/pip.conf with the internal Artifactory PyPI index:
[global]
index-url = https://artifactory.ibm.internal/artifactory/api/pypi/pypi-
virtual/simple
trusted-host = artifactory.ibm.internal
cert = /etc/ssl/certs/ibm-internal-ca-bundle.pem
timeout = 60
```

2.2 Node.js 20 LTS and Java Semeru OpenJDK 21 Configuration

- **Java Runtime Standard:** Enterprise middleware and JDBC utilities require **IBM Semeru Runtime Certified Edition OpenJDK 21 LTS** (`21.0.4+7`). Install via the internal SDK manager (`sdk install java 21.0.4-semeru`) and verify `JAVA_HOME` points to the Semeru installation directory.
- **Node.js Frontend Toolchain:** Install **Node.js v20.17.0 LTS** via `nvm` (`nvm install 20.17.0 && nvm alias default 20.17.0`) and point npm to the internal registry: `npm config set registry https://artifactory.ibm.internal/artifactory/api/npm/npm-virtual/`.

Section 3. Local Container Runtime and IBM Db2 Developer Setup

> **Standard Local IBM Db2 Development Configuration:** Local development for RAG, vector search, and transactional microservices uses the official **IBM Db2 Community Edition container (`icr.io/db2_community/db2:11.5.9.0`)** running on **Rancher Desktop** or **Podman Desktop** (Docker Desktop requires a paid commercial seat request via IT Catalog code `SW-DOCKER-PRO`). The standard local Db2 port is **`50000`** (non-TLS) and **`50001`** (TLS), default instance name is **`db2inst1`**, and the standard local RAG database name is **`DOCUSENSE`** with `PAGESIZE 32768` (32 KB) required for wide vector embedding columns.

3.1 Launching the Local IBM Db2 Container Instance

Ensure at least **8 GB of RAM** and **4 vCPUs** are allocated to your Podman or Rancher Desktop virtual machine. Create a local persistence directory (`mkdir -p ~/db2_data`) and run the following command to launch IBM Db2 locally:

```
# Pull the approved Db2 11.5.9 image from IBM Cloud Container Registry mirror:
docker pull icr.io/db2_community/db2:11.5.9.0

# Start the local Db2 container for DocuSense AI / RAG development:
docker run -h db2local \
  --name ibm-db2-dev \
  --restart=always \
  --detach \
  --privileged=true \
  -p 50000:50000 \
  -p 50001:50001 \
  -e LICENSE=accept \
  -e DB2INST1_PASSWORD="DevLocalDb2#2026" \
  -e DBNAME="DOCUSENSE" \
  -e BLU=false \
  -e ENABLE_ORACLE_COMPATIBILITY=false \
  -e UPDATEAVAIL=NO \
  -e TO_CREATE_SAMPLEDB=false \
  -e REPODB=false \
  -e IS_OSXFS=true \
  -v ~/db2_data:/database \
  icr.io/db2_community/db2:11.5.9.0
```

Wait approximately 2 to 3 minutes for the container initialization script to complete. Monitor progress by streaming the container logs with `docker logs -f ibm-db2-dev` until you see the message `Setup has completed`.

3.2 Configuring a 32 KB Page Size Bufferpool and Tablespace for Vector Embeddings

By default, standard Db2 tablespaces use a 4 KB or 8 KB page size, which can truncate or reject rows storing high-dimensional vector embeddings (such as 768-dimension or 1536-dimension floating-point arrays alongside document chunk metadata). Connect to the running container and initialize a dedicated 32 KB bufferpool (`VEC_BP32K`) and tablespace (`VEC_TS32K`):

```
# Enter the running Db2 instance shell:
docker exec -ti ibm-db2-dev su - db2inst1

# Connect to the DOCUSENSE database and provision 32 KB storage:
db2 connect to DOCUSENSE
db2 "CREATE BUFFERPOOL VEC_BP32K IMMEDIATE SIZE 2000 PAGESIZE 32 K"
db2 "CREATE LARGE TABLESPACE VEC_TS32K PAGESIZE 32 K MANAGED BY AUTOMATIC STORAGE
BUFFERPOOL VEC_BP32K"
db2 "COMMIT"
```

3.3 Installing the Python `ibm_db` Driver on macOS (Apple Silicon M1/M2/M3) and Linux

To connect Python applications (LangChain, LlamaIndex, FastAPI, or Streamlit) to IBM Db2, install the `ibm_db` and `ibm_db_sa` packages. On Apple Silicon (ARM64) macOS machines, you must export the architecture flag and ODBC driver path before running `pip install`:

```
# On macOS Apple Silicon (M1/M2/M3/M4), install prerequisite unixodbc via
Homebrew:
brew install unixodbc

# Install the IBM Db2 Python driver and SQLAlchemy adapter:
pip install --no-cache-dir ibm_db==3.2.3 ibm_db_sa==0.4.1 langchain-community

# Test local connectivity from Python using the standard DSN connection string:
#
DSN="DATABASE=DOCUSENSE;HOSTNAME=127.0.0.1;PORT=50000;PROTOCOL=TCPIP;UID=db2inst1;
PWD=DevLocalDb2#2026;"
```

3.4 Resolving Common Local Db2 Connection Errors (`SQL30081N` & `SQL1031N`)

- **Error `SQL30081N` (Communication error `0x0000006F` / Connection Refused on Port `50000`):** Indicates that the Db2 TCP/IP listener inside the container has not started or `DB2COMM` is not set to `TCPIP`. Run `docker exec -ti ibm-db2-dev su - db2inst1 -c "db2set DB2COMM=TCPIP && db2stop force && db2start"`.
- **Error `SQL1031N` (The database directory cannot be found on the indicated file system):** Occurs on macOS if the mounted host volume `~/db2_data` lacks full read/write permissions or if `IS_OSXFS=true` was omitted from `docker run`. Stop the container, run `chmod -R 775 ~/db2_data`, and recreate the container with `-e IS_OSXFS=true`.
- **Error `CLI0150E` (Driver not capable):** Occurs when passing non-UTF-8 strings to `ibm_db.exec_immediate()`. Ensure `export DB2CODEPAGE=1208` is set in your shell environment prior to launching your Python application.

Section 4. IDE Configuration, Pre-Commit Secret Scanning, and watsonx API Setup

4.1 Mandatory `detect-secrets` Pre-Commit Hook

To prevent accidental commits of IBM Cloud IAM API keys, Db2 passwords, or internal tokens, every developer must install the corporate `detect-secrets` pre-commit hook globally before making their first commit:

```
# Install IBM's detect-secrets toolchain and bind global Git hooks:
pip install pre-commit detect-secrets
git config --global init.templateDir ~/.git-templates
detect-secrets scan > .secrets.baseline
```

4.2 Local Environment Variables for IBM watsonx.ai and Db2 RAG Development

Never hardcode API keys or database credentials in Python source files. Create a local `.env` file (which must be listed in `.gitignore`) with the following standard variable names used across internal RAG reference templates:

```
# Standard .env configuration for DocuSense AI / watsonx + Db2 projects:
IBM_DB2_HOST="127.0.0.1"
IBM_DB2_PORT="50000"
IBM_DB2_DATABASE="DOCUSENSE"
IBM_DB2_USER="db2inst1"
IBM_DB2_PASSWORD="DevLocalDb2#2026"
IBM_DB2_SECURITY=" None"   # Set to "SSL" for staging/production port 50001

WATSONX_URL="https://us-south.ml.cloud.ibm.com"
WATSONX_PROJECT_ID="your-sandbox-project-uuid"
WATSONX_APIKEY="your-ibm-cloud-iam-api-key"
EMBEDDING_MODEL_ID="ibm/slate-125m-english-rtrvr-v2"
LLM_MODEL_ID="ibm/granite-3-8b-instruct"
```

Developers can request a personal sandbox `WATSONX_PROJECT_ID` and daily quota of 500,000 tokens at zero department cross-charge by visiting the internal AI Developer Portal at `w3.ibm.internal/watsonx-sandbox`.